

```
If (Money_Transfer())
    Redirect("Transaction_done.php");
```

The above code executes the money-transfer transaction, and if the transaction succeeds, then the user's browser is redirected to the *Transaction_done* page. In this case, an attacker can not only execute a CSRF attack and transfer money, but can also get feedback on whether this operation was successfully completed, using the following attack process:

1. The attacker creates an IFRAME with src= 'Transaction_done.php' with history.length element in it.
2. This alerts the attacker with the current value of the victim's history.length.
3. Now, the attacker creates another IFRAME with src to 'Money_transfer.php' again with history.length element in it.
4. If the value of history.length remains the same – then the operation was successfully completed.

However, this was a simple attack scenario; besides this, the attacker can also detect the user's authentication state, and can also access valuable intranet resources, which otherwise cannot be accessed publicly.

Cross-site user tracking

While using an IFRAME, attackers have limitations: they cannot know what the victim is doing in the IFRAME, and also, by design, an IFRAME cannot communicate with its parent page. But, using XSHM, attackers can bypass these limitations too—and in some cases, track user activities inside an IFRAME. To track the victim's activities in a page inside an IFRAME, attackers build a list of URLs that a parent page contains, or can submit. Now, whenever the victims click a link in the IFRAME, their history.length will remain the same. The URL of this link must always remain on top of the history list; here, the attacker can use *location.replace* to probe different URLs from the list, without inserting them to the top of the history list. A typical attack process is as follows:

1. The attacker creates an IFRAME with its src as the victim's site, with a history.length element in it.
2. On each load event of the IFRAME, the attacker remembers the current value of the victim's history.length, and then changes the src of the IFRAME to a URL which the parent page can access.
3. Performing the last two actions on all URLs the parent page can access, until the value of history.length ceases to increase, lets an attacker know what link a user clicked, and when it was clicked.

An example of this is a phishing attack in which the attacker opens a legitimate site from an IFRAME. As the victim clicks a link to log in, the attacker is intimated about this, and opens a fake login page instead.

Cross-site URL/parameters manipulation

This is another attack vector of XSHM, through which the

attacker can also enumerate previously browsed URLs. The attack process for this is quite simple:

1. The attacker builds an array of URLs to check (let's call it *URL[]*).
2. The attacker then runs *history.go(URL[x])* for each URL in the list.
3. If a *document.unload* occurs, the event assures the attacker that *URL[x]* was visited in the current session.

The only limitation with the above process is that the attacker must set the *URL[]* array, containing all the URLs for which the victim's visit is to be assured, prior to the attack.

Cross-site history manipulation (XSHM) is a new attack vector, by which the Same Origin Policy can be compromised, and the user's privacy can be violated. XSHM enhances CSRF by making it a two-way attack. However, the following security tips can help curb it.

Time for security

If your application uses conditional redirects, it might be vulnerable to XSHM. However, for the following code, there is no potential risk:

```
if (url != "")
    Response.Redirect(url);
```

This is because the condition *if(url != "")* is specific, and cannot produce TRUE or FALSE.

For successful prevention of cross-site history manipulation, both the URL of the origin page from which redirection is executed, and a redirected target page, should contain a random token, as shown below:

```
If ( !isAuthenticated)
    Redirect('Login.aspx?r=' + Random())
```

To prevent URL/parameters enumeration, all site URLs should contain random tokens placed inside the URL.

To learn more about these attacks, don't forget to visit the resources below. We will deal with other dangerous attacks on Web applications and Apache in the next article. Always remember: Know hacking, but no hacking. 

Further reading

- [1] http://www.owasp.org/index.php/Cross_Site_Tracing
- [2] [http://www.owasp.org/index.php/Cross_Site_History_Manipulation_\(XSHM\)](http://www.owasp.org/index.php/Cross_Site_History_Manipulation_(XSHM))

By: Arpit Bajpai

The author is a FOSS enthusiast, and loves to troubleshoot in the information security domain, while exploring new exploits and system vulnerabilities. You can send your queries and constructive feedback about the article to abajpai75 AT yahoo DOT com.